

沈阳工业大学

《操作系统》上机实验指导书

沈阳工业大学信息学院

信息科学与工程学院

实验教学中心

2026 年 3 月

目 录

一、 实验环境要求	3
1. Windows 操作系统环境要求	3
2. Linux 操作系统环境要求	3
3. 虚拟机环境 VMware (VM) 安装	3
修订	3
二、 实验内容	4
1. 实验一 Windows 系统管理	4
(1) 实验目的	4
(2) 实验环境	4
(3) 背景知识	4
(4) 实验内容与步骤	4
2. 实验二 Windows 基础进程编程	8
(1) 实验目的	8
(2) 实验环境	8
(3) 实验原理及背景知识	8
(4) 实验内容与步骤	8
3. 实验三 Windows 进程的“一生”	11
(1) 实验目的和要求	11
(2) 实验环境	11
(3) 实验原理及背景知识	11
(4) 实验内容与步骤	11
4. 实验四 进程调度与死锁	14
(1) 实验目的	14
(2) 实验环境	14
(3) 实验原理及背景知识	14
(4) 实验内容和步骤	14
5. 实验五 磁盘调度算法模拟	22
(1) 实验目的和要求	22
(2) 实验环境	22
(3) 实验原理及背景知识	22
(4) 实验内容和步骤	22
6. 实验六 操作系统调度算法	26
(1) 实验目的	26
(2) 操作系统调度算法的实现（一）	26
(3) 操作系统调度算法的实现（二）	27
(4) 基于时间片轮转的调度实现	28
7. 实验七 linux 操作系统基础	33
(1) 实验目的和要求	33
(2) 实验环境	33
(3) 背景知识	33
(4) 实验内容和步骤	33

8. 实验八 linux 操作系统线程编程	38
(1) 实验目的	38
(2) 实验环境	38
(3) 实验原理及背景知识	38
(4) 实验内容和步骤	39
9. 实验九 Linux 操作系统信号量	41
(1) 实验目的	41
(2) Linux 操作系统信号量 (一)	41
(3) Linux 操作系统信号量 (二)	42
附录	44
1. Windows 系统版本介绍	44
2. Linux 系统版本介绍	45
3. xv6 运行环境	46
4. VMware 虚拟机安装 Ubuntu	46
5. Linux 系统 c 语言编译环境	52

《操作系统》上机实验指导书

课程编号	08075211		
课程名称	操作系统	英文名称	Operating Systems
上机学时	16 学时		
先修课程	计算机组成原理、汇编语言程序设计、程序设计基础（C 语言）、面向对象程序设计（C++）、数据结构、离散数学		

一、实验环境要求

1. Windows 操作系统环境要求

操作系统：Windows 11/10 64 位 专业版（其他版本也可）

编译环境：VC++ 6.0

2. Linux 操作系统环境要求

操作系统：ubuntu 20.04（16.04 及上版本均可）

编译环境：gcc (5.3.1 及以后版本，用于编译 c 语言)，g++(5.3.1 及以后版本，用于编译 c++语言)

3. 虚拟机环境 VMware（VM）安装

建议在 Windows 系统上先安装 VMware 虚拟机或 Oracle VM VirtualBox，在虚拟机上安装 Ubuntu 操作系统，一台电脑具备多种操作系统实验环境。

虚拟机版本：VMware workstation pro 14（及以后版本）

（具体安装，参见附录 1）

修订

✧ 2023 年 5 月第一次修订。

✧ 2024 年 4 月第二次修订。

✧ 2025 年 4 月第三次修订。

增加实验六、实验九。

✧ 2026 年 3 月第四次修订。

二、实验内容

1. 实验一 Windows 系统管理

(1) 实验目的

- 1) 了解 Windows 系统管理工具及其使用；
- 2) 熟悉 Windows 系统工具的内容和应用；

(2) 实验环境

Windows 11/10/7 操作系统

(3) 背景知识

Windows 的“管理工具”中集成了许多系统管理工具，利用这些工具，管理员可以方便地实现各种系统维护和管理功能。这些工具都集中在“控制面板”的“管理工具”选项下，用户和管理员可以很容易地对它们操作和使用。在默认情况下，只有一些常用工具(如服务、计算机管理、事件查看器、数据源 (ODBC)、性能和组件服务等)随 Windows 系统的安装而安装。

- 1) 服务启动和停止由 Windows 系统提供的各项服务。
- 2) 计算机管理器管理磁盘以及使用其他系统工具来管理本地或远程计算机。
- 3) 事件查看器显示来自于 Window 和其他程序的监视与排错信息。例如，在“系统日志”中包含各种系统组件记录的事件，如使用驱动器失败或加载其他系统组件；“安全日志”中包含有效与无效的登录尝试及与资源使用有关的事件，如删除文件或修改设置等，本地计算机上的安全日志只有本机用户才能查看；“应用程序日志”中包括由应用程序记录的事件等等。

4) 性能显示系统性能图表以及配置数据日志和警报。

5) 组件服务配置并管理 COM+ 应用程序。

6) 了解 DOS 命令行窗口。

进阶，另一些工具则随系统服务的安装而添加到系统中，例如：

- 1) Telnet 服务器管理查看以及修改 Telnet 服务器设置和连接。
- 2) Internet 服务管理器管理 IIS、Internet 和 Intranet Web 站点的 Web 服务器。
- 3) 本地安全策略查看和修改本地安全策略，诸如用户权限和审计策略。

(4) 实验内容与步骤

实验时间：60 分钟。

步骤 1：登录进入 Windows 10（windows7 略有不同）。

步骤 2: 右键点击左下角“开始”，显示 windows 功能菜单，观察有哪些系统管理工具，基本功能是什么：

- 1) _____
- 2) _____
- 3) _____
- 4) _____

1、计算机管理

使用“计算机管理”可通过一个合并的桌面工具来管理本地或远程计算机，它将几个管理实用程序合并到一个控制台目录树中，使管理员可以轻松地访问特定计算机的管理属性和工具。

步骤 3: 右键“开始”选择“计算机管理”功能。

“计算机管理”使用的窗口与“Windows 资源管理器”相似。在用于导航和工具选择的控制台目录树中有“系统工具”、“存储”及“服务和应用程序”等节点，窗口右侧“名称”窗格中显示了工具的名称、类型或可用的子工具等。把系统工具、存储、服务右侧内容，填入下表。

表 1-3 实验记录

名称	类型	描述
名称	类型	描述
名称	类型	描述

2、事件查看器

事件查看器不但可以记录各种应用程序错误、损坏的文件、丢失的数据以及其他问题，

而且还可以把系统和网络的问题作为事件记录下来。管理员通过查看在事件查看器中显示的系统信息，可以迅速诊断和纠正可能发生的错误和问题。

步骤 4：在“管理工具”窗口中，双击“事件查看器”图标。

在 Windows 事件查看器中，管理员可以查看到三种类型的本地事件日志，请填入下表。

表 1-4 实验记录

名称	类型	描述	当前大小

步骤 5：在事件查看器中观察“应用程序日志”。

本地计算机中，共有_____个应用程序日志事件。

步骤 6：单击“查看”菜单中的“筛选”命令，系统日志包括的事件类型有：

3、性能监视

“性能”监视工具通过图表、日志和报告，使管理员可以看到特定的组件和应用进程的资源使用情况。利用性能监视器，可以测量计算机的性能，识别以及诊断计算机可能发生的错误，并且可以为某应用程序或者附加硬件制作计划。另外，当资源使用达到某一限定值时，也可以使用警报来通知管理员。

步骤 7：在“管理工具”窗口中，双击“性能”图标。

“性能”窗口的控制台目录树中包括的节点有：

表 1-5 实验记录

名称	描述

4、服务

步骤 8: 在“管理工具”窗口中，双击“服务”图标。

在你的本地计算机中，管理着_____个系统服务项目。

通过观察，重点描述你所感兴趣的 3 个系统服务项目：

- 1) _____
- 2) _____
- 3) _____

步骤 9: 右键“开始”->“运行”，输入命令“cmd”进入命令行窗口，测试简单的 dos 命令，help、date、dir、cd、ver、ping、tasklist、netstat、telnet 等常用 windows 命令，可以使用 HELP 显示 DOS 命令帮助。描述你感兴趣并了解的 dos 命令。

- 1) _____
- 2) _____
- 3) _____

2. 实验二 Windows 基础进程编程

(1) 实验目的

通过在 Windows 下实现 c++进程编程

- 1) 进一步熟悉操作系统的基本概念
- 2) 进一步熟悉进程的概念
- 3) 初步理解 Windows 进程的编程

(2) 实验环境

Windows 10/7 操作系统, Visual C++ 6.0 开发环境

(3) 实验原理及背景知识

- ✧ Windows 可以识别的应用程序包括控制台应用程序、GUI 应用程序和服务应用程序。
- ✧ Windows 提供了创建控制台应用程序的能力, 使用户可以利用标准的 C++ 工具, 如 iostream 库中的 cout 和 cin 对象, 来创建小型应用程序。
- ✧ 在 windows 下使用 vc++6.0 作为 c/c++开发环境简称 IDE, IDE 创建可执行程序时, 编译器将源代码编译成 OBJ 文件, 然后将其与 标准库相链接。产生的 EXE 文件是装载器指令、机器指令和应用程序的数据的集合。装载器指令告诉系统从哪里装载机器代码。另一个装载器指令告诉系统从哪里开始 执行进程的主线程。在进行某些设置后, 进入开发者提供的 main()、ServiceMain() 或 WinMain()函数的低级入口点。机器代码中包括有控制逻辑, 它所做的事包括跳 转到 Windows API 函数, 进行计算或向磁盘写入数据等。
- ✧ Windows 允许开发人员将大型应用程序分为较小的、互相有关系的服务模块, 即动态链接库(DLL)代码块, 在其中包含应用程序所使用的机器代码和应用程序的数据。

(4) 实验内容与步骤

实验时间: 30 分钟。

步骤 1: 进入 Windows, 启动 VC++ 6.0 开发环境。

步骤 2: 在“文件”菜单中选择“新建工程”, 在“工程”选项卡中选择“Win32 Console Application”, 然后在“工程名称”处输入工程名, 如“test1”, 在“位置”处选择或输入工程保存的目录。创建一个新的控制台应用程序工程。

步骤 3: 在 vc++ 左侧 classview (类视图) VS 选择 main()函数, 输入如下代码, 编译运行。

步骤 4: 程序运行的结果截图, 添加到实验报告, 并附加截图说明。

步骤 5: 观察显示结果, 了解下面两个函数的作用及输入参数。

1) 程序运行结果是什么_____

2) GetModuleFileName 函数作用_____

3) GetPriorityClass 函数的作用_____

步骤 6: 根据实验过程中程序输出的结果, 作出结论 (含遇到的各种问题及解决方案、心得以及建议等)。

```
// 显示当前进程优先级
// 教学实验中心 liangzhu
// 编译环境 vc++6.0

#include "stdafx.h"
#include<windows.h>
#include<iostream>

int main(int argc, char* argv[])
{
    //从当前过程中提取句柄
    HANDLE hProcessThis=::GetCurrentProcess ();

    //请求内核提供该过程所属的优先权类
    DWORD dwPriority=::GetPriorityClass(hProcessThis);

    //获取进程文件路径名
    TCHAR szFilename[MAX_PATH] ;
    :: GetModuleFileName(NULL, szFilename, MAX_PATH) ;

    //显示进程优先级标题
    std::cout<<"Process pathname:" << szFilename<<std::endl;

    //显示进程优先级标题
    std::cout<<"current process priority:";

    switch(dwPriority)
    {
        case HIGH_PRIORITY_CLASS:
            std::cout<<"High";
            break;
        case NORMAL_PRIORITY_CLASS:
            std::cout<<"Normal";
            break;
        case IDLE_PRIORITY_CLASS:
```

```
        std::cout<<"Idle";  
        break;  
    case REALTIME_PRIORITY_CLASS:  
        std::cout<<"Realtime";  
        break;  
    default:  
        std::cout<<"<unknow>";  
        break;  
    }  
    std::cout<<std::endl;  
  
    return 0;  
}
```

3. 实验三 Windows 进程的“一生”

(1) 实验目的和要求

通过创建进程、观察正在运行的进程和终止进程的程序设计和调试操作，熟悉操作系统的进程概念，理解 Windows 进程创建、运行、终止的“一生”。

通过阅读和分析实验程序，学习创建进程、观察进程和终止进程的程序设计方法。

(2) 实验环境

Windows 10/7 操作系统

Visual C++ 6.0 开发环境

(3) 实验原理及背景知识

Windows 所创建的每个进程都从调用 `CreateProcess()` API 函数开始，函数的任务是在对象管理器子系统内初始化进程对象，进程都以调用 `ExitProcess()` 或 `TerminateProcess()` API 函数终止。通常应用程序的框架负责调用 `ExitProcess` 函数。对于 C++ 运行库来说，这一调用发生在应用程序的 `main()` 函数返回之后。

✧ 创建进程

`CreateProcess`。调用的核心参数是可执行文件运行时的文件名及其命令行。

✧ 正在运行的进程

如果一个进程拥有至少一个执行线程，则为正在系统中运行的进程。通常这种进程使用主线程来指示它的存在。当主线程结束时，调用 `ExitProcess` API 函数，通知系统终止它所拥有的所有正在运行、准备运行或正在挂起的其他线程。当进程正在运行时，可以通过一系列的 API 函数来查看它的许多特性，其中少数特性也允许加以修改

✧ 终止进程

所有进程都是以调用 `ExitProcess()` 或者 `TerminateProcess()` 函数结束的。但最好使用前者而不要使用后者，因为进程是在完成了它所有的关闭。

`TerminateProcess()` API 函数只有打开带有 `PROCESS_TERMINATE` 访问权的进程对象，就可以终止进程，并向系统返回指定的代码。这是一种“野蛮”的终止进程的方式，但是有时却很有用。

(4) 实验内容与步骤

实验估计时间：60 分钟

步骤 1：进入 Windows，启动 VC++ 6.0 开发环境。

步骤 2：在“文件”菜单中选择“新建工程”，在“工程”选项卡中选择“Win32 Console Application”，然后在“工程名称”处输入工程名，如“proc”，在“位置”处选择或输入工程保存的目录。创建一个新的控制台应用程序工程。

步骤 3：在 vc++ 左侧 classview（类视图）VS 选择 `main()` 函数，输入和修改下面的代码，

编译运行。

步骤 4: 程序运行的结果截图, 添加到实验报告, 并附加截图说明。

步骤 5: 了解下面两个函数的作用及输入参数。

1) 把 notepad 换成 cmd, 观察输出_____

2) CreateProcess 函数作用_____

3) TerminateProcess 函数的作用_____

步骤 6: 根据实验过程中程序输出的结果, 作出结论 (含遇到的各种问题及解决方案、心得以及建议等)。

```
// 进程的“一生”, 进程管理
// 教学实验中心 liangzhu
// 编译环境 vc++6.0
#include "stdafx.h"
#include <iostream>
#include <windows.h>
#include <tlhelp32.h>
using namespace std;

bool killProcess(PROCESS_INFORMATION& processInfo){
    DWORD processId = processInfo.dwProcessId;
    PROCESSENTRY32 processEntry = {0};
    processEntry.dwSize = sizeof(PROCESSENTRY32);
    //给系统内的所有进程拍一个快照
    HANDLE handleSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS,
0);

    //遍历每个正在运行的进程
    if( Process32First(handleSnap, &processEntry) ){
        BOOL isContinue = TRUE;

        //终止子进程
        do{
            if( processEntry.th32ParentProcessID == processId ){
                HANDLE hChildProcess = OpenProcess(PROCESS_ALL_ACCESS,
FALSE, processEntry.th32ProcessID);
                if( hChildProcess ){
                    TerminateProcess(hChildProcess, 0);
                    CloseHandle(hChildProcess);
                }
            }
            isContinue = Process32Next(handleSnap, &processEntry);
        }while( isContinue );
    }
```

```

        HANDLE hBaseProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
processId);
        if( hBaseProcess ){
            TerminateProcess(hBaseProcess, 0);
            CloseHandle(hBaseProcess);
        }
    }
    DWORD exitCode = 0;
    GetExitCodeProcess(processInfo.hProcess, &exitCode);
    cout<<"exitCode="<<exitCode<<endl;
    if( exitCode == STILL_ACTIVE ){
        return false;
    }
    return true;
}

int main(){

    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory( &pi, sizeof(pi) );
    ZeroMemory( &si, sizeof(si) );
    si.cb = sizeof(si);

    if( CreateProcess(NULL, "notepad", NULL, NULL, FALSE, 0, NULL,
NULL, &si, &pi) ){
        cout<<"Notepad 进程创建、运行、关闭的一生."<<endl;
        cout<<" (1) Notepad 进程已经打开!"<<endl;
        cout<<" (2) 将在 5 秒钟后关闭 Notepad 进程!"<<endl;
        Sleep(5000);
        cout<<killProcess(pi)<<endl;
        cout<<" (3) Notepad 进程被关闭..."<<endl;
    }
    else{
        cout<<"进程创建失败!"<<endl;
        HANDLE hProcess = GetCurrentProcess();
        TerminateProcess(hProcess,0);
    }

    return 0;
}

```

4. 实验四 进程调度与死锁

(1) 实验目的

了解操作系统进程资源调度、申请、避免死锁的概念。

通过编写 C++ 程序实现银行家算法，加深对操作系统有关资源申请、避免死锁的理解。

(2) 实验环境

Windows 10/7 操作系统，Visual C++ 6.0 开发环境

(3) 实验原理及背景知识

有很多处理进程调度的算法，如：

- ❖ **先来先服务调度算法 (FCFS)**：从就绪队列的队首选择最先到达就绪队列的进程，为该进程分配
- ❖ **短进程优先调度算法 (SPF)**：从就绪队列中选择估计运行时间最短的进程，为该进程分配 CPU
- ❖ **优先权调度算法**：将 CPU 分配给就绪队列中优先权最高的进程
- ❖ **时间片轮转调度算法 (RR)**：系统将所有就绪进程按先来先服务的原则，排成一个队列，每次调度时把 CPU 分给队首进程，并令其执行一个时间片。当时间片用完时，调度程序终止当前进程的执行，并将它送到就绪队列的队尾。
- ❖ **多级队列调度算法**：将就绪队列分成多个独立队列，每个队列有自己的调度算法
- ❖ **多级反馈队列调度算法**：建立多个优先权不同的就绪队列，每个队列有大小不同的时间片

参考：<https://www.cnblogs.com/jalja/p/11432035.html>

银行家算法 (Banker's Algorithm) 是一个避免死锁 (Deadlock) 的著名算法，是由艾兹格·迪杰斯特拉在 1965 年为 T.H.E 系统设计的一种避免死锁产生的算法。它以银行借贷系统的分配策略为基础，判断并保证系统的安全运行。

我们可以把操作系统看作是银行家，操作系统管理的资源相当于银行家管理的资金，进程向操作系统请求分配资源相当于用户向银行家贷款。

为保证资金的安全，银行家规定：

- (1) 当一个顾客对资金的最大需求量不超过银行家现有的资金时就可接纳该顾客；
- (2) 顾客可以分期贷款，但贷款的总数不能超过最大需求量；
- (3) 当银行家现有的资金不能满足顾客尚需的贷款数额时，对顾客的贷款可推迟支付，但总能使顾客在有限的时间里得到贷款；
- (4) 当顾客得到所需的全部资金后，一定能在有限的时间里归还所有的资金。

参考：https://blog.csdn.net/yaopeng_2005/article/details/6935235

(4) 实验内容和步骤

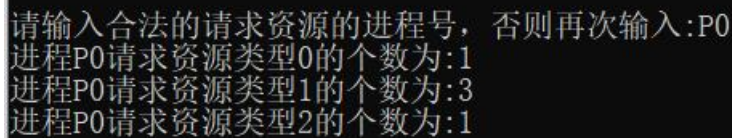
实验估计时间：60 分钟。

步骤 1：进入 Windows，启动 VC++ 6.0 开发环境。

步骤 2：在“文件”菜单中选择“新建工程”，在“工程”选项卡中选择“Win32 Console Application”，然后在“工程名称”处输入工程名，如“bank”，在“位置”处选择或输入工程保存的目录。创建一个新的控制台应用程序工程。

步骤 3：在 vc++ 左侧 classview（类视图）VS 选择 main() 函数，输入和修改下面的代码，编译运行。

步骤 4：选择不同进程 P0...P5, 并输入不同的请求资源个数，如下图，记录输出结果。



```
请输入合法的请求资源的进程号，否则再次输入:P0
进程P0请求资源类型0的个数为:1
进程P0请求资源类型1的个数为:3
进程P0请求资源类型2的个数为:1
```

步骤 5：程序运行的结果截图，添加到实验报告，并附加截图说明。

步骤 6：观察显示结果，说明下面函数的作用。

1) 观察不同的输入数据，程序运行的结果_____

2) Init() 函数作用_____

3) safeCheck() 函数的作用_____

3) show() 函数的作用_____

步骤 7：根据实验过程中程序输出的结果，作出结论（含遇到的各种问题及解决方案、心得以及建议等）。

```
// 进程调度与死锁
// 2021 年 4 月 6 日 11:27:50 liangzhu
// 教学实验中心
#include "stdafx.h"
#include <iostream>

using namespace std;

const int N = 5; ///进程数目
const int M = 3; ///资源类型数目

int AllResource[M] = {10, 5, 7}; ///各类类型资源矩阵
int Available[M]; ///可用的资源向量
int Request[M]; ///请求资源向量
int Max[N][N] =
{
    ///最大需求矩阵
    {7, 5, 3},
    {3, 2, 2},
```



```

        {9,0,2},
        {2,2,2},
        {4,3,3}
    };

    int Allocation[N][M]=
    {
        ///已分配资源矩阵
        {0,1,0},
        {2,0,0},
        {3,0,2},
        {2,1,1},
        {0,0,2}
    };

    int Need[N][N];///需求矩阵
    ///展示当前所有资源情况
    void show()
    {
        int i;
        cout<<"-----\n";
        cout<<"当前各进程需要资源[Need]数量为: \n";
        for(i=0; i<M; i++)
        {
            cout<<"\t"<<"资源"<<i;
        }
        cout<<endl;
        for(i=0; i<N; i++)
        {
            cout<<"进程 P"<<i<<": ";
            for(int j=0; j<M; j++)
            {
                cout<<"\t"<<Need[i][j];
            }
            cout<<endl;
        }
        cout<<"\n 当前各进程已分配资源[Allocation]数量为: \n";
        for( i=0; i<M; i++)
        {
            cout<<"\t"<<"资源"<<i;
        }
        cout<<endl;
        for( i=0; i<N; i++)
        {
            cout<<"进程 P"<<i<<": ";
            for(int j=0; j<M; j++)

```

```

        {
            cout<<"\t"<<Allocation[i][j];
        }
        cout<<endl;
    }
    cout<<"\n 当前各类型资源的可利用 [Available] 数量为: \n";
    cout<<"[";
    for( i=0; i<M; i++)
    {
        cout<<Available[i];
        if(i!=M-1) cout<<" ";
    }
    cout<<"]\n-----\n";
}

///检验进程号是否合法
bool checkProcessId(int id)
{
    if(id<0||id>=N) return false;
    return true;
}

///请求资源非法对应的输出提示
void invalidRequestPrint(int id)
{
    cout<<"进程 P"<<id<<"请求资源数目不合理, 请重新输入\n";
}

///资源更新
void update(int id)
{
    for(int i=0; i<M; i++)
    {
        Available[i]-=Request[i];
        Allocation[id][i]+=Request[i];
        Need[id][i]-=Request[i];
    }
}

///恢复资源更新
void rupdate(int id)
{
    for(int i=0; i<M; i++)
    {

```

```

        Available[i]+=Request[i];
        Allocation[id][i]-=Request[i];
        Need[id][i]+=Request[i];
    }
}

///安全性算法
bool safeCheck(int id)
{
    int Work[M];
    bool Finish[N];///Finish[i]=true 表示进程号为 i 的请求安全
    int safeSequence[N];///存储安全序列
    int len=0;///记录安全序列的长度
    for(int i=0; i<N; i++)
    {
        Finish[i]=false;
    }
    for(int j=0; j<M; j++)
    {
        Work[j]=Available[j];
        while(checkProcessId(id))///进程号合法
        {
            if(!Finish[id]&&Need[id][j]<=Work[j])
            {
                Work[j]+=Allocation[id][j];
                Finish[id]=true;
                safeSequence[len++]=id;
                id=0;
            }
            else id++;
        }
        for(int i=0; i<N; i++)
        {
            if(!Finish[i])
            {
                cout<<"\n 请求资源失败!!! \n\n";
                return false;
            }
        }
    }
    cout<<"\n 请求资源成功! 安全序列为: ";
    for( i=0; i<len; i++)
    {

```

```

        cout<<"P"<<safeSequence[i];
        if(i!=len-1) cout<<"->";
    }
    cout<<"\n\n";
    return true;
}

void banker()
{
    char flag='y';
    while(flag=='y' || flag=='Y')
    {
        int i=-1;
        while(!checkProcessId(i)) ///进程号非法
        {
            cout<<"请输入合法的请求资源的进程号，否则再次输入:P";
            cin>>i;
            if(!checkProcessId(i))
            {
                cout<<"非法输入，请重新输入"<<endl;
            }
        }
        for(int j=0; j<M; j++)
        {
            cout<<"进程 P"<<i<<"请求资源类型"<<j<<"的个数为:";
            cin>>Request[j];
            if(Request[j]>Need[i][j]) ///请求资源大于进程所需资源
            {
                invalidRequestPrint(i);
                flag='n';
                break;
            }
            else
            {
                ///请求资源数目合理
                if(Request[j]>Available[j]) ///可利用的资源不够用
                {
                    ///请求的该资源类型的数目大于其可用资源数目
                    invalidRequestPrint(i);
                    flag='n';
                    break;
                }
            }
        }
    }
}

```

```

        if(flag=='y' || flag=='Y')
        {
            update(i); ///资源更新
            if(!safeCheck(i))
            {
                ///安全性算法,处于非安全状态
                rupdate(i); ///恢复资源更新以便输出
            }
            else
            {
                ///确认分配请求资源后能否释放资源(即收回全部分配的资源)
                bool flag=1;
                for(int j=0; j<M; j++)
                {
                    if(Need[i][j]!=0)
                    {
                        flag=0;
                        break;
                    }
                }
                if(flag)
                {
                    for(int j=0; j<M; j++)
                        Available[j]+=Allocation[i][j];
                }
            }
        }
        show();
        cout<<"按'y'或'Y'继续请求资源分配,否则退出: ";
        cin>>flag;
    }
}

///系统资源的初始化
void init()
{
    ///初始化可利用资源向量
    int tmp;
    for(int j=0; j<M; j++)
    {
        tmp=AllResource[j];
        for(int i=0; i<N; i++)
        {
            tmp-=Allocation[i][j];
            if (tmp>0)

```

```

        Available[j]=tmp;
    else
        Available[j]=0;
    }
}
///初始化需求矩阵
for(int i=0; i<N; i++)
{
    for(int j=0; j<M; j++)
    {
        Need[i][j]=Max[i][j]-Allocation[i][j];
    }
}
}

int main()
{
    init();
    show();
    banker();
    return 0;
}

```

5. 实验五 磁盘调度算法模拟

(1) 实验目的和要求

通过对磁盘调度算法的设计，深入理解提高磁盘访问速度的原理。

(1) 设计一个磁盘调度模拟程序，从而使磁盘调度算法更加形象化，容易使人理解，使磁盘调度的特点更简单明了；

(2) 加深对先来先服务算法、最短寻道时间优先算法、扫描算法以及循环扫描算法等磁盘调度算法的理解；

(3) 针对给定的磁盘访问序列，运行各种调度算法得出调度过程

(4) 算法所需的各种参数由输入产生（手工输入或者随机数产生）

(2) 实验环境

Windows 10/7 操作系统 Visual C++ 6.0 开发环境

(3) 实验原理及背景知识

常见的磁盘调度算法有：

- ❖ FCFS：先来先服务算法；
- ❖ SSTF：最短寻道时间算法；
- ❖ SCAN：扫描算法（也叫电梯调度算法）；
- ❖ CSCAN：循环扫描算法

（详细介绍自行网上学习）

(4) 实验内容和步骤

实验估计时间：60 分钟。

步骤 1：进入 Windows，启动 VC++ 6.0 开发环境。

步骤 2：在“文件”菜单中选择“新建工程”，在“工程”选项卡中选择“Win32 Console Application”，然后在“工程名称”处输入工程名，如“disk”，在“位置”处选择或输入工程保存的目录。创建一个新的控制台应用程序工程。

步骤 3：在 vc++ 左侧 classview（类视图）VS 选择 main() 函数，输入和修改下面的代码，编译运行。

步骤 4：程序运行的结果截图，添加到实验报告，并附加截图说明。

步骤 5：观察显示结果，了解各算法优势。

- 1) 程序运行结果说明什么_____
- 2) 那个算法平均寻道时间最短_____
- 3) SSTF 算法的优势_____

步骤 6：根据实验过程中程序输出的结果，作出结论（含遇到的各种问题及解决方案、心得以及建议等）。

```

// 磁盘调度算法模拟
// 教学实验中心 liangzhu
// 编译环境 vc++6.0
#include "stdafx.h"
#include<iostream>
#include<cmath>
#include <ctime>
using namespace std;

/*1、FCFS 算法*/
void FCFS(int *data,int n,int *order){
    for(int i=0;i<=n;i++){
        order[i]=data[i];
    }
}

/*2、SSTF 算法*/
void SSTF(int *data,int n,int *order) {
    for(int i=0;i<=n;i++) {
        order[i]=data[i];
    }
    for(int j=1;j<=n;j++){
        int p=j;
        for(int k=j+1;k<=n;k++){
            if(abs(order[k]-order[j-1])<abs(order[p]-order[j-1])) {
                p=k;
            }
        }
        if(p!=j) {
            int temp=order[j]; order[j]=order[p]; order[p]=temp; }
    }
}

/*3、SCAN 算法*/
void SCAN(int *data,int n,int *order) {
    int j=1; int q=n; order[0]=data[0];
    for(int i=1;i<=n;i++){
        if(data[i]>order[0]) {order[j]=data[i];j++;}
        else{order[q]=data[i];q--;}
    }
    for(int z=1;z<j;z++) {
        int p=z;
        for(int k=z+1;k<j;k++){if(order[k]<order[p]) { p=k; } }
        if(p!=z){ int temp=order[z]; order[z]=order[p]; order[p]=temp; }
    }
}

```



```

    }
    for(int h=j;h<=n;h++) {
        int p=h;
        for(int k=h+1;k<=n;k++) { if(order[k]>order[p]) { p=k;} }
        if(p!=h) { int temp=order[h]; order[h]=order[p];
order[p]=temp; }
    }
}

/*4、CSCAN 算法*/
void CSCAN(int *data,int n,int *order) {
    int j=1; int q=n; order[0]=data[0];
    for(int i=1;i<=n;i++) {
        if(data[i]>order[0]) {order[j]=data[i];j++;}
        else {order[q]=data[i];q--;}
    }
    for(int z=1;z<j;z++) {
        int p=z;
        for(int k=z+1;k<j;k++) { if(order[k]<order[p]) { p=k;} }
        if(p!=z) { int temp=order[z]; order[z]=order[p];
order[p]=temp; }
    }
    for(int h=j;h<=n;h++) {
        int p=h;
        for(int k=h+1;k<=n;k++) { if(order[k]<order[p]) { p=k;} }
        if(p!=h) { int temp=order[h]; order[h]=order[p];
order[p]=temp; }
    }
}

/*5、主函数*/
void main()
{
    int data[100]; data[0]=100;
    int n=0;//要寻访磁道个数
    srand((int)time(0)); n=rand()%20+5;
    cout<<"模拟磁道:"<<n<<endl;
    for(int i=1;i<=n;i++) {
        data[i]=rand()%200+1;
        cout<<data[i]<<" ";
    }
    cout<<endl;
    cout<<"-----"<<endl;
    int order[100]; int length=0;
    FCFS(data,n,order); cout<<"FCFS 算法"<<endl;
}

```

```

    for(int j=1;j<=n;j++)
    { length+=abs(order[j]-order[j-1]); cout<<order[j]<<" "; }
cout<<endl;
cout<<"平均寻道长度"<<length*1.0/n<<endl;
SSTF(data,n,order); cout<<"SSTF 算法"<<endl; length=0;
for(int g=1;g<=n;g++)
{ length+=abs(order[g]-order[g-1]); cout<<order[g]<<" "; }
cout<<endl;
cout<<"平均寻道长度"<<length*1.0/n<<endl;
SCAN(data,n,order); cout<<"SCAN 算法"<<endl; length=0;
for(int d=1;d<=n;d++)
{ length+=abs(order[d]-order[d-1]); cout<<order[d]<<" "; }
cout<<endl;
cout<<"平均寻道长度"<<length*1.0/n<<endl;
CSCAN(data,n,order); cout<<"CSCAN 算法"<<endl; length=0;
for(int k=1;k<=n;k++)
{ length+=abs(order[k]-order[k-1]); cout<<order[k]<<" "; }
cout<<endl;
cout<<"平均寻道长度"<<length*1.0/n<<endl;
}

```

6. 实验六 操作系统调度算法

(1) 实验目的

通过课程学习，已经掌握了基本的调度算法，比如先来先服务、短作业优先、高优先权优先、高相应比优先、轮转等，本次编程作业内容就是在一个仿真系统环境下实现这些算法。

(2) 操作系统调度算法的实现（一）

假定我们的系统每个任务有四个用于调度的信息，包括作业的 ID，作业到达时间，作业需要运行的时间，作业的优先级。我们用 csv 格式的文本文件作为作业集的输入，格式如下图所示：

```
[root@localhost lec3]# cat example.txt
#id,arrive_time,burst_time,priority
1,1,3,2
2,0,4,4
3,3,2,0
4,2,5,1
5,4,4,3
```

在该文件中，各个属性用逗号（英文逗号）隔开，依次对应作业的 ID，作业到达时间，作业需要运行的时间，作业的优先级。

程序启动后，将读取该文件，并按照到达时间顺序存放在一个链表中，链表的结构定义为：

```
typedef struct job{
    int id;           //job id
    int arrive_time;  //the time when the job arrives.
    int burst_time;   //the duration the job need to execute.
    int priority;     // 0~10,0 is Highest priority.
    int handled;
    struct job *next;
} jobs;
```

其中 handled 是一个标识位，表示该作业是否已经被处理。

读取文件并生成作业链表的代码已经实现，为了简化，实现中未考虑作业文件的数据是否合规，要求作业文件中的每项作业的四个属性都是整数。

```
typedef struct workflow{
    int id;
    int start_time;
    int end_time;
    struct workflow *next;
}workflows;
```

按照调度算法，对输入的作业集将形成执行序列，我们定义一个结构体来保存作业的执行序列：

其中 id 为作业 id，start_time 为作业开始执行时间；end_time 为作业完成的时间。

注意：1) Workflow 形成一个链表，链表的顺序就是执行的顺序。

如果一个作业是分多次执行完，那么在链表中其 id 将出现多次，总执行时间是作业的 burst_time。

输出

调度算法在作业集上调度执行完成后将形成一个 workflow 链表，将该链表进行输出就是作业执行的顺序。

```
void printworkflows(workflows *head){
    workflows *p=head;
    while(p){
        printf("ID:%d,start_time:%d,end_time:%d\n",p->id,p->start_time,p->end_time);
        p = p -> next;
    }
}
```

实验要求：根据所提供的程序，补充 TODO 部分，完成 FIFO 和 SJF 调度算法。

(3) 操作系统调度算法的实现（二）

通过实验一，同学们掌握了先来先服务、短作业优先模拟实现的方法，那么本次实验将在实验一的基础上完成优先权调度和时间片轮转调度算法的实现。

基于优先权的调度实现

优先权调度就是为作业分配一个优先权，可分为静态优先权和动态优先权。所谓静态优先权就是为作业分配优先权后将不再变化，动态优先权是作业的优先权随着运行不断的变化。本次任务是实现静态优先权调度算法。另外，优先权调度算法又可分为抢占型和非抢占型，本次要求实现非抢占型。（抢占型可以再完成基于时间片轮转调度算法后自行实现）

为此，我们为作业设定一个优先权的变量，如下图中的 priority，假定优先权从 0 到 10，数值小代表优先权高（思考为什么这样？）

```
typedef struct job{
    int id;           //job id
    int arrive_time;  //the time when the job arrives.
    int burst_time;   //the duration the job need to execute.
    int priority;     // 0~10,0 is Highest priority.
    int handled;
    struct job *next;
} jobs;
```

实验要求：请结合上次作业实现的先来先服务和短作业优先，设计实现。（提示：短作业优先也是一种优先权）。

(4) 基于时间片轮转的调度实现

之前实现的先来先服务，短作业优先和非抢占的优先权调度算法都有一个共同特点，那就是一个作业一旦被调度执行，那么就会将该作业执行结束后再执行下一项作业。而时间片轮转调度算法则不同，一个作业被调度执行，执行完时间片后，会停止执行，然后排队等待下次调度到自己才继续执行，这样就需要有个排队机制。其基本思想是：“系统将所有就绪进程按先来先服务原则排成一个队列；每次调度时，把 CPU 分配给队首进程，当其执行完一个时间片时，调度程序据中断信号停止该进程的执行，并将它送往就绪队列队尾；然后把处理机分配给就绪队列中新的队首进程”。那么这里还有一个细节问题，就是一个作业正在执行，这时候来了一个新作业，该将该新作业放在队列的首部还是尾部？

请大家根据自己的理解实现基于时间片轮转的算法（提示：需要思考如何实现排队）

(5) 任务代码

```
/*
dispatcher.c
Create by yanghb from SUT,2023.
Used for lec3 of Operating System.
*/
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct job{
    int id;           //job id
    int arrive_time;  //the time when the job arrives.
    int burst_time;   //the duration the job need to execute.
    int priority;     // 0~10,0 is Highest priority.
    int handled;
    struct job *next;
} jobs;
typedef struct workflow{
    int id;
    int start_time;
    int end_time;
    struct workflow *next;
}workflows;
jobs *readjobs(char *filename){
    FILE *fp;
    char str[64];
    char *p;
    jobs *job,*prev,*temp;
    jobs *head = NULL;
    fp = fopen(filename,"r");
```

```

while(fgets(str,sizeof(str),fp)!=NULL){
    if(strlen(str)>1){
        if(strncasecmp(str,"#",1)==0)
            continue;
        job = (jobs *)malloc(sizeof(jobs));
        if(job!=NULL){
            memset(job,0,sizeof(jobs));
            p = strtok(str,",");
            if(p!=NULL)
                job->id = atoi(p);
            p = strtok(NULL,",");
            if(p!=NULL)
                job->arrive_time = atoi(p);
            p = strtok(NULL,",");
            if(p!=NULL)
                job->burst_time = atoi(p);
            p = strtok(NULL,",");
            if(p!=NULL)
                job->priority = atoi(p);

            prev = NULL;
            temp = head;
            while((temp!=NULL)&&(temp->arrive_time < job->arrive_time)){
                prev = temp;
                temp = temp->next;
            }
            if(prev == NULL)
                head = job;
            else
                prev->next = job;
            job->next = temp;
        }
    }
    fclose(fp);
    return head;
}

void printjobs(jobs *head){
    jobs *p;
    p = head;
    while(p){
        printf("%d,%d,%d,%d\n",p->id,p->arrive_time,p->burst_time,p->priority);
        p = p->next;
    }
}

```

```

    }
}
void printworkflows(workflows *head){
    workflows *p=head;
    while(p){
        printf("ID:%d,start_time:%d,end_time:%d\n",p->id,p->start_time,p->end
_time);
        p = p -> next;
    }
}
int parser_alg(char *alg){
    if(alg == NULL)
        return -1;
    if(strcasecmp(alg,"fifo")==0)
        return 1;
    if(strcasecmp(alg,"sjf")==0)
        return 2;
    return 0;
}
void free_workflows(workflows *head){
    workflows *p = head;
    while(p){
        head = head->next;
        free(p);
        p = head;
    }
}
void free_jobs(jobs *head){
    jobs *p = head;
    while(p){
        head = head->next;
        free(p);
        p = head;
    }
}
void fifo(jobs *head){
    int now_time = 0;
    jobs *p,*q;
    workflows *wfhead=NULL,*wf,*wfp;
    p = head;
    while(p){
        wf = (workflows *)malloc(sizeof(workflows));
        memset(wf,0,sizeof(workflows));
        /*TODO: implement the FIFO ALG*/
    }
}

```

```

    /*-----*/
    p = p->next;
}
    printworkflows(wfhead);
    free_workflows(wfhead);
}
int shift_time(jobs *head,int now_time){
    jobs *p = head;
    while(p){
        if(p->handled == 0){
            if(now_time <= p->arrive_time)
                return p->arrive_time;
            else
                return now_time;
        }
        p = p->next;
    }
    return now_time;
}
jobs *sjf_get_nextjob(jobs *head,int now_time){
    jobs *p = head;
    jobs *prev=NULL;
    while(p){
        /*TODO: implement the sjf get nextjob*/
        /*-----*/

        p = p->next;
    }
    return prev;
}
void sjf(jobs *head){
    int now_time =0;
    jobs *p,*q;
    workflows *wfhead=NULL,*wf,*wfp;
    now_time = shift_time(head,now_time);
    while(p = sjf_get_nextjob(head,now_time)){
        p->handled = 1;
        wf = (workflows *)malloc(sizeof(workflows));
        memset(wf,0,sizeof(workflows));
        /*TODO: implement the SJF ALG*/
        /*-----*/

    }
    printworkflows(wfhead);
}

```



```

    free_workflows(wfhead);
}
int main(int argc, char *argv[]){
    char filename[64];
    int alg;
    jobs *head;

    if(argc != 3){
        printf("Usage: ./dispatcher <job_filename> <alg>\n");
        printf("alg: fifo sjf\n");
        return -1;
    }
    head = readjobs(argv[1]);
    printjobs(head);
    alg = parser_alg(argv[2]);
    switch(alg){
case 1:
        fifo(head);
        break;
case 2:
        sjf(head);
        break;
default:
        printf("No such algorithm\n");
    }
    free_jobs(head);
    return 0;
}

```

完整代码参考附件。

7. 实验七 linux 操作系统基础

(1) 实验目的和要求

通过本实验，以 ubuntu 操作系统为例，了解操作系统的安装过程；
学习 linux 操作的常见命令；
学习在 ubuntu 操作系统中安装软件包的方式；
在 ubuntu 中搭建 gcc 开发环境；
学会在 ubuntu 系统中进行 c 语言开发，编译，运行过程；
学习利用 C 语言在 ubuntu 中创建进程 fork 函数和创建线程的函数，实现课程 PPT 上创建进程和线程的例子。

通过本实验，

(2) 实验环境

Ubuntu 20.04 操作系统 g++ 编译环境(g++ 5.4.0 20160609)

(3) 背景知识

Linux，全称 GNU/Linux，是开源的类 UNIX 操作系统，基于 POSIX 的多用户、多任务、支持多线程和多 CPU 的操作系统。它能运行主要的 Unix 工具软件、应用程序和网络协议。支持 32 位和 64 位硬件，是一个性能稳定的多用户网络操作系统。Linux 有上百种不同的发行版，如基于社区开发的 debian、archlinux，和基于商业开发的 Red Hat Enterprise Linux、SUSE、Oracle Linux 等。

Ubuntu 是一个以桌面应用为主的 Linux 操作系统，基于 Debian 发行版和 Gnome 桌面环境。具体特点可参考附录或互联网检索。

(4) 实验内容和步骤

实验估计时间：60 分钟。

步骤 1：安装虚拟机（vmware 或者 Oracle VM VirtualBox）

1) 在虚拟机上安装 ubuntu 系统（22.04.2 LTS）

① 下载地址：<https://cn.ubuntu.com/download/desktop>.

(2) 学习在 ubuntu 上安装软件包的方法

① 配置 apt 源（阿里源，清华源等）

② 学习用 apt-get 安装软件的方法

要求学生熟练掌握 Linux 各种文件操作命令，包括：

使用控制字符执行特殊功能；

使用 file 和 strings 命令确定文件类型；

使用 cat 和 more 命令显示文本文件的内容；

使用 head 和 tail 命令显示文本文件的部分内容；

使用 wc 命令确定单词、行和字符数；

使用 diff 命令比较 2 个文件；

回顾文件和目录命名约定；

使用 touch 命令创建新文件；

使用 mkdir 命令创建 新目录；

使用 rm 命令删除文件。

步骤 1: 进入 Ubuntu 桌面 (Gnome) 系统.

✓ 观察 Gnome 主要的界面及功能，了解和掌握基本操作。

步骤 3: 使用 ctrl+alt+T 打开 terminal 终端，练习使用表一所列命令，

1) 显示当前文件夹内容_____

2) 新建文件夹: testdir_____

3) 在 testdir 文件夹里新建 test.txt 文件_____

4) _____

表一：目录及文件常用命令

命令	解释
ls	显示某一个目录下的内容（文件和目录）
ls -a	显示所有的文件
ls -l	文件详细信息显示
cp	复制文件
cp -r	复制目录
mv	移动文件或目录位置
touch	新建文件
mkdir	新建目录
*rm	删除文件
*rm -r	删除目录（慎重使用）
rmdir	删除空目录
cat	查看文件内容
more	查看长文件内容

步骤 4: 在 terminal 终端下练习使用表二所列命令:

- 1) 当前在线用户名称_____
- 2) 系统硬盘信息_____
- 3) 当前系统进程信息_____
- 4) 系统剩余内存情况_____

表二: 常用系统信息查询命令

命令	解释
ps aux	查看系统的进程
w	显示在线用户信息
top	显示系统任务信息, 包括 cpu、内存等
date	显示系统日期和时间
uptime	显示系统运行时间
df	查看系统硬盘信息
free	查看系统内存信息
whoami	显示当前正在进行操作的用户名

步骤 5: 安装 gcc 开发环境,编译、运行 **lec1** 中的 **fork** 程序和 **lec2** 中的**创建线程程序**。
在 terminal 终端下输入 gedit 打开文本编辑器,输入下面的文件,保存为 test.cpp 文件。

```
//test1.cpp
#include <iostream>
using namespace std;

int main()
{
    int space, rows;

    cout <<"Input rows: ";
    cin >> rows;
    if (rows>20){
        cout<<"To long!!!max 20 rows"<<endl;
        rows=20;
    }
}
```

```

    }

    for(int i = 1, k = 0; i <= rows; ++i, k = 0)
    {
        for(space = 1; space <= rows-i; ++space)
        {
            cout << " ";
        }

        while(k != 2*i-1)
        {
            cout << "* ";
            ++k;
        }
        cout << endl;
    }
    return 0;
}

```

步骤 5: 在 terminal 终端下练习使用表三所列命令:

- 1) 列出包含“cpp”的文件_____
- 2) 列当前文件夹所有包含 main 文件的行_____
- 3) 显示 test1.cpp 文件的行数、字数_____

表三: 文件操作命令

命令	解释	用法
grep string	过滤文本	和其他命令连用, 如 <code>ls -l grep string</code> 为只显示具有 string 的行, <code>cat 文件名 grep string</code> 为只显示文件中具有 string 的行
grep -n string	过滤文本, 把所找到的行在行前加上行号列出	用法与 grep 类似, 加上参数即可 <code>ls -l grep -n string</code>
grep string -r	搜索文本	用法: <code>grep string -r 目录名</code> 在给定目录下搜索所有文件中含有 string 的行
grep -v string	过滤文本, 把不包含给定 string 的行列出	用法与 grep 类似, 加上参数即可 <code>ls -l grep -v string</code>

wc	统计文件，给出文件的 行数、字数、字符数	用法：wc 文件名
chmod	改变文件权限	chmod mode 文件名

步骤 6：在 terminal 终端下练习使用表三所列命令：

- 1) Ping 一个 ip 地址，写出延迟毫秒数_____
- 2) 写出本机的 IP 地址_____
- 3) 5 秒钟后关机_____

表四：其他常用命令

命令	解释
passwd	修改密码
ifconfig	查看本地网络配置
ping	查看与对方机器是否连接
exit	退出登录系统
ctrl+C	中止程序执行
ctrl+D	退出系统
kill -9 进程 id	强制杀死进程
netstat -a	查看网络连接状态
shutdown	关机命令 参数：-t 后面加上秒数，表示“过几分钟关机”，shutdown -t 10

8. 实验八 linux 操作系统线程编程

(1) 实验目的

“生产者消费者”问题是一个著名的同时性编程问题的集合。通过编写经典的“生产者 消费者”问题的实验，读者可以进一步熟悉 Linux 中多线程编程，并且掌握用信号量处理线程间的同步互斥问题。

(2) 实验环境

Ubuntu 20.04 desktop, g++ 编译环境(g++ 5.4.0 20160609)s

(3) 实验原理及背景知识

生产者消费者问题 (Producer-consumer problem)，也称有限缓冲问题 (Bounded-buffer problem)，是一个多线程同步问题的经典案例。该问题描述了两个共享固定大小缓冲区的线程，即所谓的“生产者”和“消费者”在实际运行时会发生的问题。生产者的主要作用是生成一定量的数据放到缓冲区中，然后重复此过程。与此同时，消费者也在缓冲区消耗这些数据。该问题的关键就是要保证生产者不会在缓冲区满时还要生产和加入数据，消费者也不会会在缓冲区中空时去获取和消耗数据。

❖ 缓冲区

解决问题首先需要设置大小为 N 的缓冲区，提供给生产者和消费者线程使用。生产者产生物品放到缓冲区，消费者将物品从缓冲区移走。

互斥关系：

缓冲区的访问是互斥的，包括生产者和生产者之间、消费者和消费者之间、生产者和消费者之间。

同步关系：

当缓冲区满时生产者进程需要等待，当缓冲区空时消费者进程需要等待。

❖ 信号量

用信号量解决生产者消费者问题。

互斥：

有节缓冲区是一个临界资源，对临界资源得访问需要设置一个信号量 `mutex`

同步：

设置两个同步信号量 `empty` 和 `full`，其初值分别为 n 、 0 ，用于临界资源得具体数目，同时也可以表示等待条件

具体原理如下：

有一个有限缓冲区和两个线程：生产者和消费者。

他们分别把产品放入缓冲区和从缓冲区中拿走产品。当一个生产者在缓冲区满时必须等待，当一个消费者在缓冲区空时页必须等待。

(1) 信号量的考虑 这里使用 2 个信号量，其中两个信号量 `blank_number` 和

product_number 分别用于解决生产者和消费者线程之间的同步问题。

(4) 实验内容和步骤

实验估计时间：60 分钟。

步骤 1：进入 Ubuntu 桌面（Gnome），在 home 目录下新建文件 prod.cpp 程序文件。

步骤 2：使用 gedit 打开 prod.cpp 文件，输入下面的程序代码，并保存。

步骤 3：使用 ctrl+alt+T 打开终端，输入编译命令：

```
g++ -std=c++11 prod.cpp -lpthread
```

步骤 4：编译生成 prod.out 执行文件，在终端上输入 ./prod.out 运行，程序运行的结果截图，添加到实验报告，并附加截图说明。

步骤 5：观察输出的结果。

1) 程序运行结果演示了_____

2) 改大 producer 的 usleep 参数，产生什么效果_____

3) 改大 consumer 的 usleep 参数，产生什么效果_____

步骤 6：根据实验过程中程序输出的结果，作出结论（含遇到的各种问题及解决方案、心得以及建议等）。

```
// 信号量实现 生产者 消费者
// 教学实验中心 liangzhu
// 编译环境 linux gcc++
// 编译: g++ -std=c++11 prod.cpp -lpthread
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <semaphore.h>

#define NUM 10 //最大缓冲区属
int queue[NUM];
sem_t blank_number, product_number; //空闲空间信号量, 产品信号量
void *producer(void *arg)
{
    int i = 0;
    while (1) {
        sem_wait(&blank_number); //p 操作 生产者将空格子数
        //为 0 则阻塞等待
        queue[i] = rand() % 1000 + 1; //生产一个产品
        printf("生产者: 产品%d 置于空间[%d]\n", queue[i], i);
        sem_post(&product_number); //v 操作 将产品数++
    }
}
```



```

        i = (i+1) % NUM;                                //借助下标实现环形
        usleep(rand()%1000000);
    }
}

void *consumer(void *arg)
{
    int i = 0;

    while (1) {
        sem_wait(&product_number);                        //消费者将产品数--,为0则阻塞
        等待
        printf(" --- 消费者: 取出产品%d 从空间[%d]\n", queue[i], i);
        queue[i] = 0;                                    //消费一个产品
        sem_post(&blank_number);                          //消费掉以后,将空闲空间数++

        i = (i+1) % NUM;
        usleep(rand()%1000000);
    }
}

int main(int argc, char *argv[])
{
    srand((unsigned)time(NULL));
    for(int i=0;i<=10;i++) printf("%d ", rand()%100);
    pthread_t pid, cid;

    sem_init(&blank_number, 0, NUM);                      //初始化空闲空间信号量为10
    sem_init(&product_number, 0, 0);                      //产品数为0

    pthread_create(&pid, NULL, producer, NULL);
    pthread_create(&cid, NULL, consumer, NULL);

    pthread_join(pid, NULL);
    pthread_join(cid, NULL);

    sem_destroy(&blank_number);
    sem_destroy(&product_number);

    return 0;
}

```

9. 实验九 Linux 操作系统信号量

(1) 实验目的

多线程编程是一种并发编程模型，它利用计算机的多核处理器或多处理器架构，使得一个程序可以同时执行多个线程，广泛应用于图形界面程序、网络编程、IO 编程、游戏开发等场景。在课堂上我们讲解了如何通过 Pthread 库在 Linux 下创建多线程。

(2) Linux 操作系统信号量（一）

下图所示代码是一个例子，在该例子中，创建三个线程 a,b,c。

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

pthread_t a_thread, b_thread, c_thread;

void *func1(void *arg) {
    printf("a_thread is starting \n");
    sleep(2);
    printf("a_thread finished \n");
}

void *func2(void *arg) {
    printf("b_thread is starting... \n");
    sleep(2);
    printf("b_thread finished \n");
}

void *func3(void *arg) {
    printf("c_thread is starting...\n");
    sleep(2);
    printf("c_thread finished \n");
}

int main() {

    pthread_create(&a_thread, NULL, func1, NULL);
    pthread_create(&b_thread, NULL, func2, NULL);
    pthread_create(&c_thread, NULL, func3, NULL);
    pthread_join(a_thread, NULL);
    pthread_join(b_thread, NULL);
    pthread_join(c_thread, NULL);
    return 0;
}
```

(1) 编译并运行该程序，注意编译的时候需要使用库 pthread，示例：

```
gcc -o main main.c -lpthread
```

请在实际环境多次运行该程序，观察并记录这 3 个线程开始运行及结束运行的顺序在每

次执行过程中是否一致。

Linux 在 pthread 库中提供了信号量机制。使用方法如下：

使用前需要先定义信号量，信号量的数据类型为 sem_t,定义方法：

sem_t semPtr; // 其中 sem_t 为数据类型，semPtr 为信号量变量名称。

使用 sem_init 函数初始化信号量，sem_init 的原型为：

```
int sem_init(sem_t *sem,int pshared,unsigned int value);
```

其中 pshared 控制信号量的类型，如果其值为 0，就表示这个信号量是当前进程的局部信号量，否则信号量就可以在多个进程之间共享。value 为信号量的初值。

使用 sem_wait 函数和 sem_post 函数进行信号量的 PV 操作，原型分别为：

```
int sem_wait(sem_t *sem); //P 操作
```

```
int sem_post(sem_t *sem); //V 操作
```

当程序结束不在需要该信号量的时候执行 sem_destroy 函数，释放信号量，其原型为：

```
int sem_destroy(sem_t *sem);
```

请使用信号量实现 a,b,c 三个线程按照如下同步方式进行执行：

b->a->c(b 执行完了才能执行 a，a 执行完了才能执行 c)。

a,c->b(a,c 可并发执行，但必须等 a,c 都执行完了才能执行 b)。

(3) Linux 操作系统信号量 (二)

假设我们定义一个全局变量 int buff;作为一个缓冲区，线程 a 和 b 是生产者，生产者会向 buff 写一个数据，c 是消费者，从 buff 中取走数据。线程 a,b,c 都被修改成了无限循环程序，如下图中程序所示。

实验要求：

(1) 编译并运行该程序，查看并记录运行结果，是否能实现 a 和 b 生产的数据用且只被 c 用一次。

(2) 请使用信号量实现 a 和 b 生产的数据用且只被 c 用一次（生产者消费者模型）。

```

#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

int buff;
pthread_t a_thread, b_thread, c_thread;

void *func1(void *arg) {
    printf("a_thread is starting \n");
    while(1){
        buff = rand();
        printf("a_thread produce:%d\n",buff);
    }
    printf("a_thread finished \n");
}

void *func2(void *arg) {
    printf("b_thread is starting... \n");
    while(1){
        buff = rand();
        printf("b_thread produce:%d\n",buff);
    }
    printf("b_thread finished \n");
}

void *func3(void *arg) {
    printf("c_thread is starting...\n");
    while(1){
        printf("c_thread consume:%d\n",buff);
    }
    printf("c_thread finished \n");
}

int main() {
    srand(time(0));
    pthread_create(&a_thread, NULL, func1, NULL);
    pthread_create(&b_thread, NULL, func2, NULL);
    pthread_create(&c_thread, NULL, func3, NULL);
    pthread_join(a_thread, NULL);
    pthread_join(b_thread, NULL);
    pthread_join(c_thread, NULL);
    return 0;
}

```

附录

1. Windows 系统版本介绍

(1) windows 7 系统（老版本，目前使用较少了）

一共有六个版本，分别是：Starter（初级版）、Home Basic（家庭普通版）、Home Premium（家庭高级版）、Professional（专业版）、Enterprise（企业版）以及 Ultimate（旗舰版），推荐使用专业版。

❖ Starter（初级版）

功能最少的一个版本，连很多基本的常用功能都没有，比如只支持 32 位处理器，不支持 64 位处理器；没有 win7 最好用的 windows 媒体中心和移动中心；就连基本桌面背景更换，都有限制。它并不是给个人用户使用的，这个系统似乎只是用于非常低端的一些设备，而且只有部分特定的硬件才能安装它。

❖ Home Basic（家庭普通版）

对 Aero 特效有限制；没有 windows 媒体中心、远程桌面、家庭网络组。

❖ Home Premium（家庭高级版）

家庭高级版，基本的功能都有，不支持 windows 域、XP 模式、多语言。很多个人购买的笔记本，台式机，预装的正版系统，就是这个版本。

❖ Professional（专业版）

最常见的版本，常用功能都有，比如远程桌面、网络备份、位置感知打印、演示模式、活动目录和域支持、加密文件系统等。64 位的专业版系统，支持高达 192G 的内存。很多厂商的电脑或者笔记本，预装了专业版。

❖ Enterprise（企业版）

面向企业的一个版本，多出了很多只有企业级功能，比如：数据管理、数据共享、数据安全；UNIX 应用支持、BitLocker 驱动器加密、分支缓存等。

❖ Ultimate（旗舰版）

旗舰版和企业版同样的功能。区别是在授权方式和一小部分应用服务上有所不同。

(2) Windows 10 系统

win10 的版本有：家庭版、专业版、企业版、教育版、LCSC 版、工作站版，依照功能大小分布来说：家庭版<专业版<企业版=教育版<工作站版，LCSC 版独立。

❖ 家庭版

针对家用电脑的版本，很多个人购买的台式机或者笔记本，预装了这款系统。相比较 win7，win10 的家庭版很是厚道，常用的功能比如虚拟桌面、应用商店、Cortana 娜娜语音助手、Xbox One 流媒体游戏、微软 Windows Hello、虹膜、指纹登录等，都是有的，没有过分的删掉这些最基本的常用功能。

❖ 专业版

专业版针对的是电脑发烧友或者技术类人员，除了具有家庭版所有功能外，主要多出了安全类和办公室类的功能，比如允许用户管理设备及应用、支持远程及移动生产力场景、保护敏感企业数据、云技术支持等，还有一些增强型技术，比如组策略、远程访问服务、Bitlocker 驱动器加密、域名连接等。

❖ 企业版

针对是企业级别，功能最全，除了拥有专业版所有功能，多出比如支持应用白名单的 AppLocker、通过点对点连接与其他 PC 共享下载与更新的 BranchCache 以及基于组策略控制的开始屏幕、无需 VPN 即可连接的 Direct Access 等。

❖ 教育版

面向学术机构提供的版本啦。教育版的功能除了没有 long term servicing branch 功能和企业版几乎没有任何差别，这个功能是用来让系统只更新安全补丁，不更新功能补丁。

❖ 工作站版

Windows10 工作站版，是 win10 的最高版本，可以支持 4 个 cpu 和 6T 的内存。具有“卓越性能模式”，其他版本都没有这个功能。它可以让系统实现动态的性能优化，通过自动识别 cpu 和 gpu 的使用占用量，不断自动调节，让电脑系统达到并一直保持最高性能。另外，它还支持 RDMA（远程直接内存互访），可以通过 InfiniBand 直接访问远程主机的内存。此外，工作站版和其他版本还有一个不同，就是安装完的系统非常干净，没有你不喜欢的各种东西。

（如果电脑性能可以的话，安装工作站版是较好的选择）

2. Linux 系统版本介绍

目前 Linux 有上百个不同的版本，比较流行的有 RedHat、CentOS、Ubuntu、Debian 等。

（1）CentOS 系统

CentOS 是 RedHat 的社区版本，可以兼容 RedHat 的 RPM 安装包，一般用作服务器，预安装的也都是服务器的内容。除了没有 RedHat 服务外，用起来与 RedHat 相差不大，包管理一般用 rpm 或者 yum 来管理。

（2）Ubuntu 系统

Ubuntu 是基于 Debian 发行版和 GNOME 桌面环境，也有 KDE 版本，属于较流行的发行版，它强化了图形方面的功能，有面向一般用户的桌面版本，也有服务器版本。因为其桌面版本比较漂亮，有较多多媒体方面的软件和 ubuntu 软件中心的支持，对一般用户也还算好用，所以较流行。其服务器版本也就没什么好说了，都差不多。包管理一般用 apt-get。

（3）Linux 系统选择

如果做服务器，选则 CentOS 或者 Ubuntu Server 版；如果用作桌面系统，选择用 Ubuntu desktop。

3. xv6 运行环境

(1) 下载 ubuntu 操作系统安装包

可以到清华大学镜像站进行下载:

<https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/>

20.04 版本链接如下。

<https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/22.04/ubuntu-22.04.5-desktop-amd64.iso>

(2) 安装虚拟机

可以是 vmware 或 Oracle VM VirtualBox, 也可以是 windows 自带的 Hyper-V。如果你的操作系统是 windows 的 HOME 版本, 想使用 Hyper-V 的话, 需要先安装 Hyper-V 管理器。方法可以参考如下: 把下面的代码, 放到一个文本编辑器软件里面, 保存为 InstallHyperV.cmd, 然后在资源管理器里面, 鼠标右键点它, 在下拉菜单里面选择以管理员身份运行。

```
pushd "%~dp0"
dir /b %SystemRoot%\servicing\Packages\*Hyper-V*.mum >hyper-v.txt
for /f %%i in ('findstr /i . hyper-v.txt 2^>nul') do dism /online /norestart
/add-package:"%SystemRoot%\servicing\Packages\%%i"
del hyper-v.txt
Dism /online /enable-feature /featurename:Microsoft-Hyper-V-All /LimitAccess /ALL
Pause
```

(3) 安装 xv6 系统环境

在 ubuntu 执行如下指令

```
sudo apt-get update
sudo apt-get install git build-essential gdb-multiarch qemu-system-misc
gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
git clone git://g.csail.mit.edu/xv6-labs-2020
cd xv6-labs-2020/
git checkout util
make qemu
那么 xv6 就运行起来了。
```

4. VMware 虚拟机安装 Ubuntu

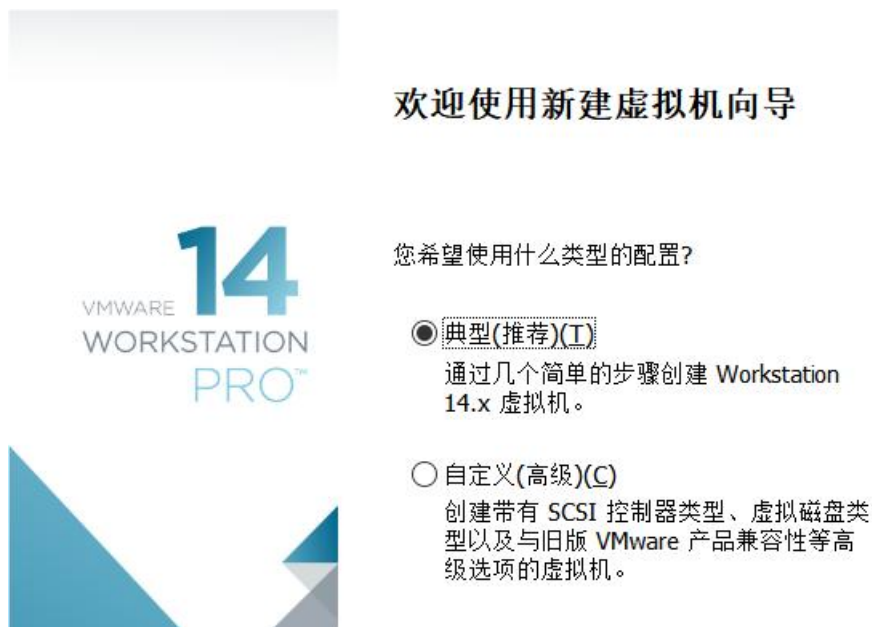
(1) 准备工作:

❖ 已安装 VMware workstation pro 环境, 本例使用的版本是 14.02。

下载 Ubuntu desktop 的 ISO 安装镜像文件, 以 16.04 版本为例 (自行安装可使用 20

以上版本）。

(2) 创建虚拟机：菜单选择“文件”->“创建虚拟机”，使用创建向导，选择典型配置方式安装，点击下一步，特殊需求可选择自定义安装。



(3) 选择要安装的客户机系统，选择准备好的 Ubuntu 映像文件（ubuntu-16.04-desktop-amd64.iso），VMware 会自动识别选择的操作系统类型，并指导以后的安装，点击下一步。

安装客户机操作系统

虚拟机如同物理机，需要操作系统。您将如何安装客户机操作系统？

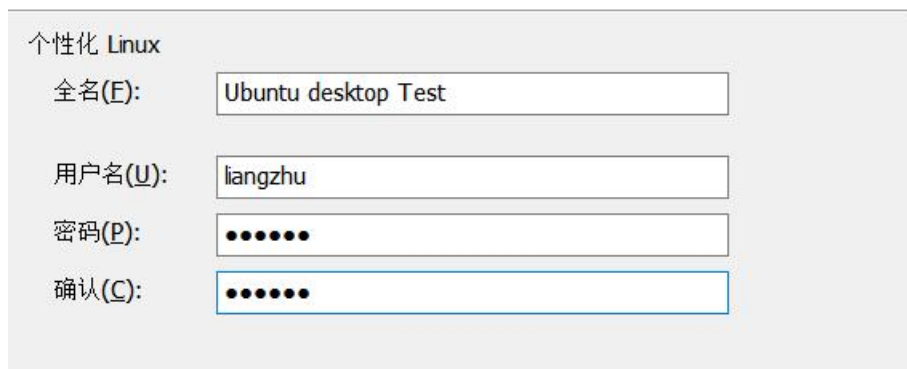


(4) 自定义安装系统的全名，用户名和密码输入自己常用的用户名和密码，注意用户使用

英文，（linux 默认系统管理员是 root），完成后点击下一步。

简易安装信息

这用于安装 Ubuntu 64 位。



个性化 Linux

全名(E): Ubuntu desktop Test

用户名(U): liangzhu

密码(P): ●●●●●●

确认(C): ●●●●●●

（5）输入虚拟机名称，这个名称是在 VMware 左侧一览中显示的名称；设置安装的目录，此目录是虚拟机安装的位置，设置完成后点击下一步。

命名虚拟机

您希望该虚拟机使用什么名称？



虚拟机名称(V):

Ubuntu 64 位

位置(L):

D:\Virtual Machines Files\Ubuntu 64 位

浏览(R)...

在“编辑”>“首选项”中可更改默认位置。

（6）指定安装虚拟机的磁盘容量，容量可以默认为 20G，以后在使用过程中不够可以扩充；选择虚拟机存储的硬盘位多文件还是单文件，使用默认值就可以，在 windows10 下影响不大。

指定磁盘容量

磁盘大小为多少?

虚拟机的硬盘作为一个或多个文件存储在主机的物理磁盘中。这些文件最初很小，随着您向虚拟机中添加应用程序、文件和数据而逐渐变大。

最大磁盘大小 (GB)(S):

针对 Ubuntu 64 位 的建议大小: 20 GB

☐ 将虚拟磁盘存储为单个文件(O)

☒ 将虚拟磁盘拆分成多个文件(M)

拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。

(7) 显示按向导配置配置的详细信息，没有问题可点击完成开始安装，需要修改配置可以选择退回去修改相应的配置。

注意：如果安装过程出现错误请查看是否是 32 位或 64 位操作系统的问题，或者电脑是否已经在 Bios 里设置支持虚拟机。

已准备好创建虚拟机

单击“完成”创建虚拟机，并开始安装 Ubuntu 64 位 和 VMware Tools。

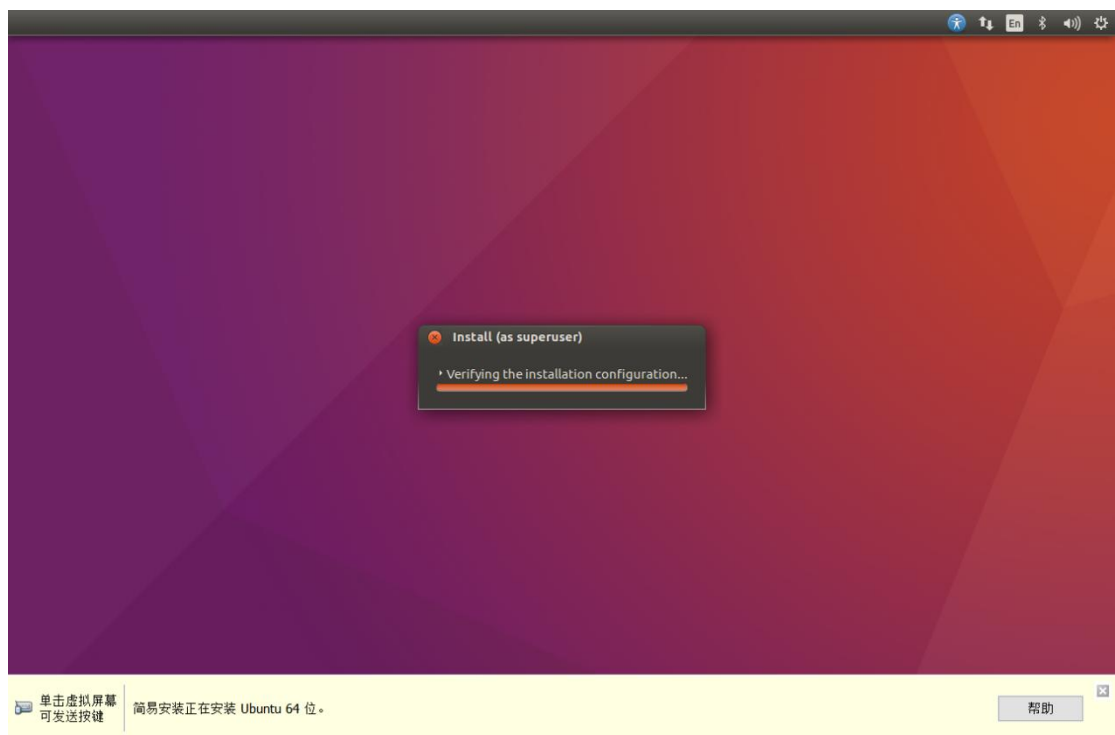
将使用下列设置创建虚拟机:

名称:	Ubuntu 64 位
位置:	D:\Virtual Machines Files\Ubuntu 64 位
版本:	Workstation 14.x
操作系统:	Ubuntu 64 位
硬盘:	20 GB, 拆分
内存:	1024 MB
网络适配器:	NAT
其他设备:	CD/DVD, USB 控制器, 打印机, 声卡

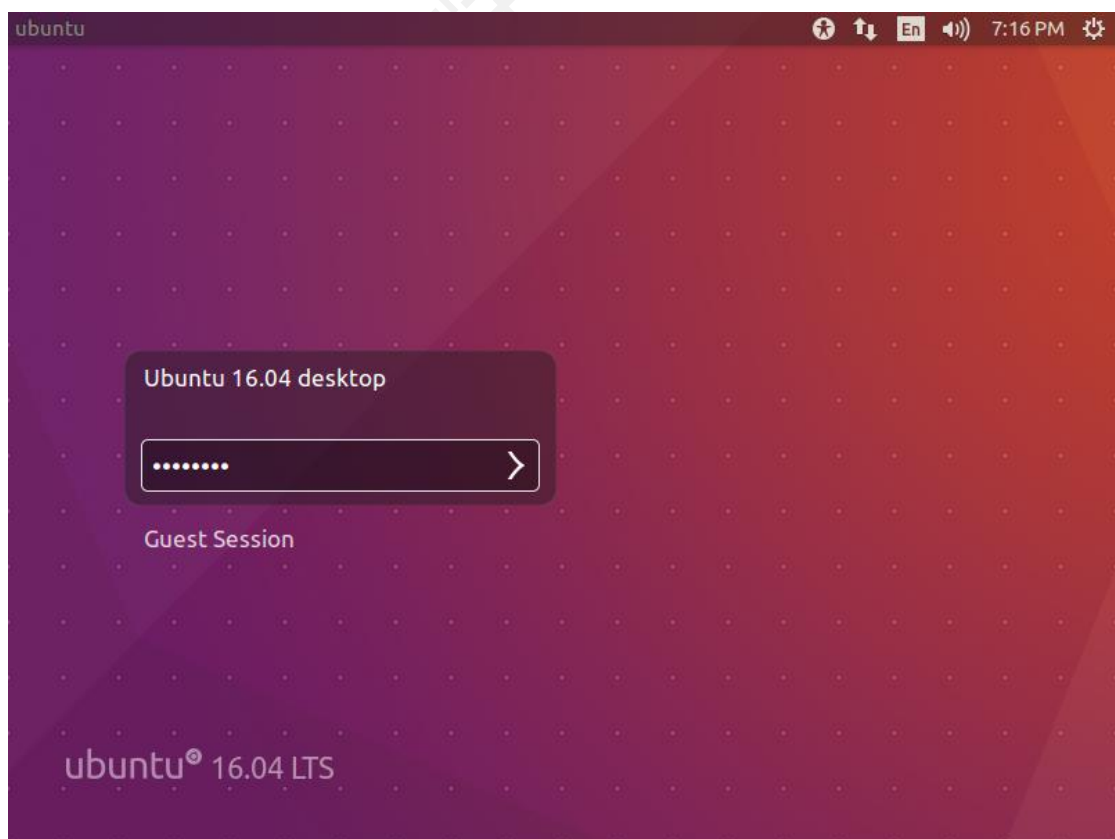
[自定义硬件\(C\)...](#)

☒ 创建后开启此虚拟机(P)

(8) VMware 下正式开始安装 Ununtu 操作系统。

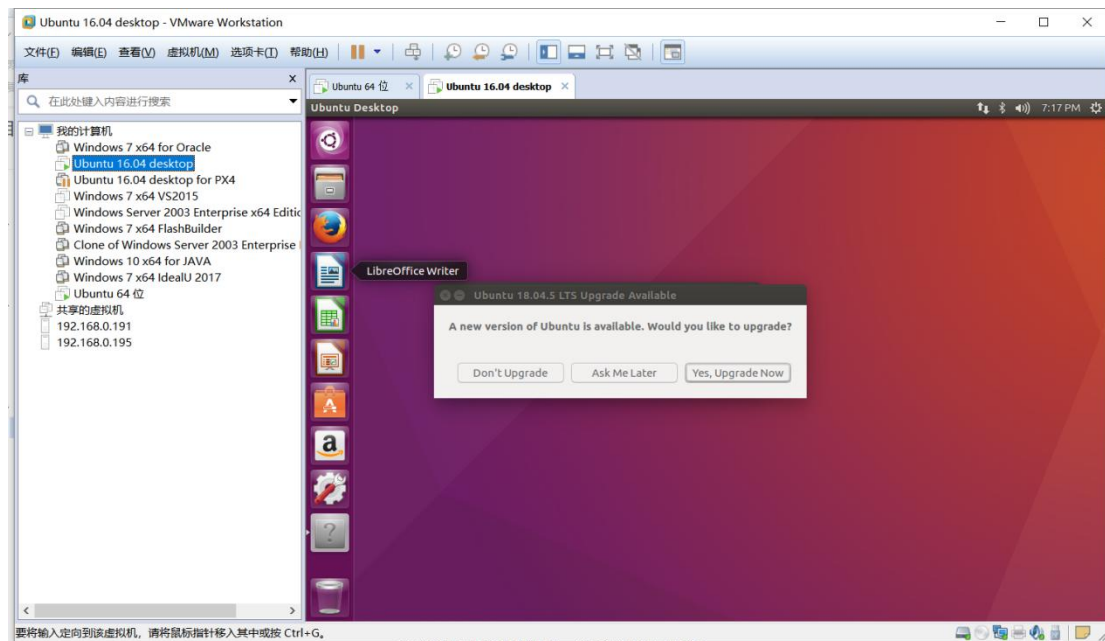


(9) 安装完成后，输入前面设置的密码进入 ubuntu 操作系统界面。

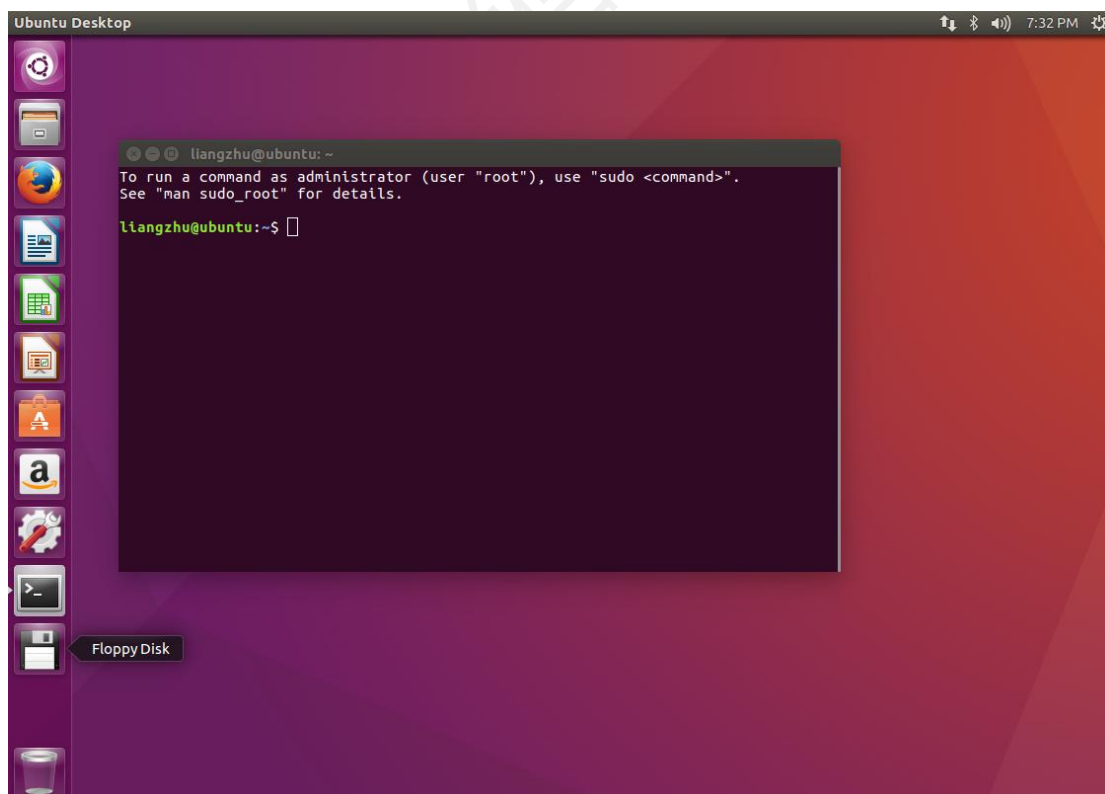


(10) 如下图是 VMware 进入 Ubuntu 操作系统后样子，弹出的窗口是提示有新版本可以更

新，视情况选择，可以使用 VMware 的全屏模式，这样跟独立系统一样的感觉，而且可以随时切回 windows 系统，超级方便。



(11) 安装完成!!!!!!!!!!!!，可以使用 **ctrl+alt+t** 打开桌面终端。



(Ubuntu 安装全文完)

5. Linux 系统 c 语言编译环境

(1) Ubuntu 默认安装了 gcc 和 g++ 环境, 在 linux 系统下 gcc 是 c 语言命令行模式下编译器, g++ 是 c++ 命令行模式下编译器。

(2) 使用 ctrl+alt+t 打开命令行终端, 在命令提示符下输入下面命令查看版本:

```
gcc --version
```

```
g++ --version
```

(3) 如果没有编译环境, 需要安装 gcc 和 g++, Ubuntu 使用如下命令安装:

```
# 版本安装: 这里选择的式版本 5
```

```
sudo apt-get install gcc-5
```

```
sudo apt-get install g++-5
```

(4) 编译环境可以使用 vim, gedit, vs code 等, 我们使用 Ubuntu desktop 默认安装的 gedit, 在命令窗口输入 gedit, 在 gedit 下输入下面的程序, 保存程序, 文件名 hello.cpp。

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Hello, world!" << endl;
    cout << "Good morning, 中国!" << endl;
    return 0;
}
```

(5) 用 g++ 编译, linux 系统下编译后输出的执行文件默认为 a.out, 可使用 -o 参数指定文件名:

```
g++ hello.cpp          默认输出 a.out 可执行文件
```

(6) 执行 .out 文件输出结果, 注意不能直接输入 a.out, 需要在前面加上路径名, 如下:

```
./a.out
```

(7) 输出结果如下:

```
liangzhu@ubuntu:~$ g++ hello.cpp
liangzhu@ubuntu:~$ ./a.out
Hello, world!
Good morning, 中国!
liangzhu@ubuntu:~$
```

(编译环境 全文完)

6. 调度算法完整代码参考

```
/*
dispatcher.c
Create by yanghb from SUT,2023.
Used for lec3 of Operating System.
*/
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct job{
    int id;           //job id
    int arrive_time;  //the time when the job arrives.
    int burst_time;   //the duration the job need to execute.
    int priority;     // 0~10,0 is Highest priority.
    int handled;
    struct job *next;
} jobs;
typedef struct workflow{
    int id;
    int start_time;
    int end_time;
    struct workflow *next;
}workflows;
jobs *readjobs(char *filename){
    FILE *fp;
    char str[64];
    char *p;
    jobs *job,*prev,*temp;
    jobs *head = NULL;
    fp = fopen(filename,"r");
    while(fgets(str,sizeof(str),fp)!=NULL){
        if(strlen(str)>1){
            if(strncasecmp(str,"#",1)==0)
                continue;
            job = (jobs *)malloc(sizeof(jobs));
            if(job!=NULL){
                memset(job,0,sizeof(jobs));
                p = strtok(str,",");
                if(p!=NULL)
                    job->id = atoi(p);
                p = strtok(NULL,",");
                if(p!=NULL)
                    job->arrive_time = atoi(p);
                p = strtok(NULL,",");
            }
        }
    }
}
```

```

    if(p!=NULL)
        job->burst_time = atoi(p);
    p = strtok(NULL, ",");
    if(p!=NULL)
        job->priority = atoi(p);

    prev = NULL;
    temp = head;
    while((temp!=NULL)&&(temp->arrive_time < job->arrive_time)){
        prev = temp;
        temp = temp->next;
    }
    if(prev == NULL)
        head = job;
    else
        prev->next = job;
    job->next = temp;
}
}
fclose(fp);
return head;
}
void printjobs(jobs *head){
    jobs *p;
    p = head;
    while(p){
        printf("%d,%d,%d,%d\n",p->id,p->arrive_time,p->burst_time,p->priority)
;
        p = p->next;
    }
}
void printworkflows(workflows *head){
    workflows *p=head;
    while(p){
        printf("ID:%d,start_time:%d,end_time:%d\n",p->id,p->start_time,p->end
_time);
        p = p -> next;
    }
}
int parser_alg(char *alg){
    if(alg == NULL)
        return -1;
    if(strcasecmp(alg, "fifo")==0)

```

```

    return 1;
if(strcasecmp(alg,"sjf")==0)
    return 2;
if(strcasecmp(alg,"pri_p")==0)
    return 3;
if(strcasecmp(alg,"pri_np")==0)
    return 4;
if(strcasecmp(alg,"rr")==0)
    return 5;
if(strcasecmp(alg,"multi")==0)
    return 6;
}
void free_workflows(workflows *head){
    workflows *p = head;
    while(p){
        head = head->next;
        free(p);
        p = head;
    }
}
void free_jobs(jobs *head){
    jobs *p = head;
    while(p){
        head = head->next;
        free(p);
        p = head;
    }
}
void fifo(jobs *head){
    int now_time = 0;
    jobs *p,*q;
    workflows *wfhead=NULL,*wf,*wfp;
    p = head;
    while(p){
        wf = (workflows *)malloc(sizeof(workflows));
        memset(wf,0,sizeof(workflows));
        wf->id = p->id;
        wf->start_time = p->arrive_time>now_time?p->arrive_time:now_time;
        wf->end_time = wf->start_time + p->burst_time;
        now_time = wf->end_time;
        if(wfhead ==NULL)
            wfhead = wf;
        else{
            wfp = wfhead;

```



```

    while(wfp->next!=NULL)
        wfp = wfp->next;
    wfp->next = wf;
}
p = p->next;
}
printworkflows(wfhead);
free_workflows(wfhead);
}
int shift_time(jobs *head,int now_time){
    jobs *p = head;
    while(p){
        if(p->handled == 0){
            if(now_time <= p->arrive_time)
                return p->arrive_time;
            else
                return now_time;
        }
        p = p->next;
    }
    return now_time;
}
jobs *sjf_get_nextjob(jobs *head,int now_time){
    jobs *p = head;
    jobs *prev=NULL;
    while(p){
        if(p->handled==0 && p->arrive_time<=now_time){
            if(prev == NULL)
                prev = p;
            else{
                if(p->burst_time < prev->burst_time)
                    prev = p;
            }
        }
        p = p->next;
    }
    return prev;
}
void sjf(jobs *head){
    int now_time =0;
    jobs *p,*q;
    workflows *wfhead=NULL,*wf,*wfp;
    now_time = shift_time(head,now_time);
    while(p = sjf_get_nextjob(head,now_time)){

```

```

        p->handled = 1;
        wf = (workflows *)malloc(sizeof(workflows));
        memset(wf,0,sizeof(workflows));
        wf->id = p->id;
        wf->start_time = p->arrive_time>now_time?p->arrive_time:now_time;
        wf->end_time = wf->start_time + p->burst_time;
        now_time = wf->end_time;
        now_time = shift_time(head,now_time);
        if(wfhead ==NULL)
            wfhead = wf;
        else{
            wfp = wfhead;
            while(wfp->next!=NULL)
                wfp = wfp->next;
            wfp->next = wf;
        }
    }
    printworkflows(wfhead);
    free_workflows(wfhead);
}

jobs *pri_get_nextjob(jobs *head,int now_time){
    jobs *p = head;
    jobs *prev=NULL;
    while(p){
        if(p->handled==0 && p->arrive_time<=now_time){
            if(prev == NULL)
                prev = p;
            else{
                if(p->priority < prev->priority)
                    prev = p;
            }
        }
        p = p->next;
    }
    return prev;
}

void pri_np(jobs *head){
    int now_time =0;
    jobs *p,*q;
    workflows *wfhead=NULL,*wf,*wfp;
    now_time = shift_time(head,now_time);
    while(p = pri_get_nextjob(head,now_time)){
        p->handled = 1;
        wf = (workflows *)malloc(sizeof(workflows));

```

```

memset(wf,0,sizeof(workflows));
wf->id = p->id;
wf->start_time = p->arrive_time>now_time?p->arrive_time:now_time;
wf->end_time = wf->start_time + p->burst_time;
now_time = wf->end_time;
now_time = shift_time(head,now_time);
if(wfhead ==NULL)
    wfhead = wf;
else{
    wfp = wfhead;
    while(wfp->next!=NULL)
        wfp = wfp->next;
    wfp->next = wf;
}
}
printworkflows(wfhead);
free_workflows(wfhead);
}
typedef struct readylist{
    jobs *runningjob;
    int runtime;
    struct readylist *next;
}readies;
void pri(jobs *head){

}
void rr(jobs *head){
    int timeslic = 2;
    int run_time;
    readies *rhead=NULL,*rtail=NULL,*r=NULL,*rp;
    workflows *wfhead=NULL,*wftail=NULL,*wf,*wfp;
    int now_time=0;
    jobs *p;
    p = head;
    if(p != NULL)
        now_time = p->arrive_time;

    while(1){
        while(p&&(p->handled == 0)&&(p->arrive_time<=now_time)){
            rp = (readies *)malloc(sizeof(readies));
            memset(rp,0,sizeof(readies));
            rp->runningjob = p;
            p->handled = 1;
            if(rtail==NULL)

```

```

    rhead = rp;
else
    rtail->next = rp;
rtail = rp;
p = p->next;
}

if(r!=NULL){
    if(rtail == NULL)
        rhead = r;
    else
        rtail->next = r;
    rtail = r;
}
// run
if(rhead!=NULL){
    r = rhead;
    rhead = rhead->next;
    if(rhead == NULL)
        rtail = NULL;
    r->next = NULL;

    run_time = r->runningjob->burst_time - r->runtime;
    if(run_time > timeslic)
        run_time = timeslic;
    r->runtime+=run_time;
    wf = (workflows *)malloc(sizeof(workflows));
    memset(wf,0,sizeof(workflows));
    wf->id = r->runningjob->id;
    wf->start_time = now_time;
    now_time = wf->end_time = now_time + run_time;

    if(wftail==NULL)
        wfhead = wf;
    else
        wftail->next = wf;
    wftail = wf;
    if(r->runtime >= r->runningjob->burst_time){
        free(r);
        r = NULL;
    }
}else{
    now_time = shift_time(head,now_time);
}

```

```

    if(p==NULL && rhead == NULL)
        break;

}
printworkflows(wfhead);
    free_workflows(wfhead);
}
int main(int argc, char *argv[]){
    char filename[64];
    int alg;
    jobs *head;

    if(argc != 3){
        printf("Usage: ./dispatcher <job_filename> <alg>\n");
        printf("alg: fifo sjf pri_p pri_np rr rp\n");
        return -1;
    }
    head = readjobs(argv[1]);
    printjobs(head);
    alg = parser_alg(argv[2]);
    switch(alg){
case 1:
    fifo(head);
    break;
case 2:
    sjf(head);
    break;
case 3:
    pri(head);
    break;
case 4:
    pri_np(head);
    break;
case 5:
    rr(head);
    break;
default:
    printf("No such algorithm\n");
    }
    free_jobs(head);
    return 0;
}

```

7. 线程同步代码参考

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

pthread_t a_thread, b_thread, c_thread;
sem_t ba,ac;

void *func1(void *arg) {
    sem_wait(&ba);
    printf("a_thread is starting \n");
    sleep(2);
    printf("a_thread finished \n");
    sem_post(&ac);
}

void *func2(void *arg) {
    printf("b_thread is starting... \n");
    sleep(2);
    printf("b_thread finished \n");
    sem_post(&ba);
}

void *func3(void *arg) {
    sem_wait(&ac);
    printf("c_thread is starting...\n");
    sleep(2);
    printf("c_thread finished \n");
}

int main() {
    sem_init(&ba,0,0);
    sem_init(&ac,0,0);
    pthread_create(&a_thread, NULL, func1, NULL);
    pthread_create(&b_thread, NULL, func2, NULL);
    pthread_create(&c_thread, NULL, func3, NULL);
    pthread_join(a_thread, NULL);
    pthread_join(b_thread, NULL);
    pthread_join(c_thread, NULL);
    sem_destroy(&ba);
    sem_destroy(&ac);
    return 0;
}
```

8. 生产者消费者参考代码

```
#include <pthread.h>
#include <semaphore.h>
```

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
int buff;
pthread_t a_thread, b_thread, c_thread;
// 定义信号量
sem_t empty;    // 表示缓冲区是否为空
sem_t full;     // 表示缓冲区是否有数据
sem_t mutex;    // 互斥锁, 保护对缓冲区的访问
void *func1(void *arg) {
    printf("a_thread is starting \n");
    while(1) {
        sem_wait(&empty);
        sem_wait(&mutex);

        buff = rand();
        printf("a_thread produce:%d\n", buff);

        sem_post(&mutex);
        sem_post(&full);
    }
    printf("a_thread finished \n");
}
void *func2(void *arg) {
    printf("b_thread is starting... \n");
    while(1) {
        sem_wait(&empty);
        sem_wait(&mutex);

        buff = rand();
        printf("b_thread produce:%d\n", buff);

        sem_post(&mutex);
        sem_post(&full);
    }
    printf("b_thread finished \n");
}
void *func3(void *arg) {
    printf("c_thread is starting...\n");
    while(1) {
        sem_wait(&full);
        sem_wait(&mutex);
    }
}

```

```

        printf("c_thread consume:%d\n", buff);

        sem_post(&mutex);
        sem_post(&empty);
    }
    printf("c_thread finished \n");
}

int main() {
    srand(time(0));

    // 初始化信号量
    sem_init(&empty, 0, 1); // 初始时缓冲区为空
    sem_init(&full, 0, 0); // 初始时没有数据
    sem_init(&mutex, 0, 1); // 互斥锁初始为可用

    pthread_create(&a_thread, NULL, func1, NULL);
    pthread_create(&b_thread, NULL, func2, NULL);
    pthread_create(&c_thread, NULL, func3, NULL);

    pthread_join(a_thread, NULL);
    pthread_join(b_thread, NULL);
    pthread_join(c_thread, NULL);

    // 销毁信号量
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}

```